

模型能力

上下文缓存

 Copy page

💡 上下文缓存功能通过缓存重复的上下文内容来显著降低 Token 消耗和响应延迟。当您在对话中重复使用相同的系统提示词或历史对话时，缓存机制会自动识别并复用这些内容，从而提升性能并降低成本。

功能特性

- **自动缓存识别**：隐式缓存，智能识别重复的上下文内容，无需手动配置
- **显著降低成本**：缓存命中的 Token 按更低价格计费，大幅节省成本
- **提升响应速度**：减少重复内容的处理时间，加快模型响应
- **透明化计费**：详细显示缓存命中的 Token 数量，响应字段
`usage.prompt_tokens_details.cached_tokens`
- **广泛兼容性**：支持所有主流模型，包括 GLM-4.6、GLM-4.5 系列等

上下文缓存通过对输入的消息内容进行计算并识别出与之前请求中相同或高度相似的内容。当检测到重复内容时，系统会复用之前的计算结果，从而避免重复计算这些内容所需的 Token。

这种机制特别适用于以下场景：

- **系统提示词复用**：在多轮对话中，系统提示词通常保持不变，缓存可以显著降低这部分的 Token 消耗。
- **重复任务**：对于一致的指令进行多次处理相似内容的任务，缓存可以提高效率。
- **多轮对话历史**：在复杂的对话中，历史消息往往包含大量重复信息，缓存可以有效降低这部分的 Token 使用。

基础缓存示例

```
# 第一次请求 - 建立缓存
curl --location 'https://open.bigmodel.cn/api/paas/v4/chat/completions' \
--header 'Authorization: Bearer YOUR_API_KEY' \
--header 'Content-Type: application/json' \
--data '{
  "model": "glm-4.7",
  "messages": [
    {
      "role": "system",
      "content": "你是一个专业的数据分析师，擅长解释数据趋势和提供业务洞察。"
    },
    {
      "role": "user",
      "content": "如何分析用户留存率？"
    }
  ]
}'
```

复用缓存示例



```
curl --location 'https://open.bigmodel.cn/api/paas/v4/chat/completions' \
--header 'Authorization: Bearer YOUR_API_KEY' \
--header 'Content-Type: application/json' \
--data '{
  "model": "glm-4.7",
  "messages": [
    {
      "role": "system",
      "content": "你是一个专业的数据分析师，擅长解释数据趋势和提供业务洞察。"
    },
    {
      "role": "user",
      "content": "什么是漏斗分析？"
    }
  ]
}'
```

响应包含上下文缓存的 Token 使用信息：

```
{
  "usage": {
    "prompt_tokens": 1200,
    "completion_tokens": 300,
    "total_tokens": 1500,
    "prompt_tokens_details": {
      "cached_tokens": 800
    }
  }
}
```

最佳实践

推荐：使用稳定的系统提示词

```
system_prompt = """
```

你是一个专业的技术顾问，具有以下特点：

- 深厚的技术背景和丰富的项目经验
- 能够提供准确、实用的技术建议
- 善于用简洁明了的语言解释复杂概念

请根据用户问题提供专业的技术指导。

```
"""
```

应用场景



多轮对话

- 智能客服系统
- 个人助理服务



批量处理

- 代码审查批处理
- 内容批量分析



模板化应用

- 报告生成模板
- 标准化流程处理



教育培训

- 作业批改辅助
- 学习资料解析

注意事项



- 完全相同的内容缓存命中率最高
- 轻微的格式差异可能影响缓存效果
- 缓存有合理的时效性，过期后会重新计算

计费说明

上下文缓存采用差异化计费策略：

- 新内容 Token：按标准价格计费
- 缓存命中 Token：按优惠价格计费（通常为标准价格的 50%）
- 输出 Token：按标准价格计费

计费示例：

假设标准价格为 0.01 元/1K Token：

请求详情：

- 总输入 Token：2000
- 缓存命中 Token：1200
- 新内容 Token：800
- 输出 Token：500

计费计算：

- 新内容费用： $800 \times 0.01/1000 = 0.008$ 元
- 缓存费用： $1200 \times 0.005/1000 = 0.006$ 元
- 输出费用： $500 \times 0.01/1000 = 0.005$ 元
- 总费用：0.019 元

相比无缓存 ($2500 \times 0.01/1000 = 0.025$ 元)，节省 24%

